

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

**Fraud Detection and Risk Scoring System Using Apache
Spark and Machine Learning**

by

RocketUpTeam

AN Duong Binh

TUYEN Le Quang

LONG Pham Duc

TRUNG Do Quoc

QUAN Duong Hong

NHI Nguyen Le Hong

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

© Copyright by RocketUpTeam 2026

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

**Fraud Detection and Risk Scoring System Using Apache
Spark and Machine Learning**

by

RocketUpTeam

AN Duong Binh

TUYEN Le Quang

LONG Pham Duc

TRUNG Do Quoc

QUAN Duong Hong

NHI Nguyen Le Hong

A thesis submitted in conformity with the requirements

for the degree of Master of Software Engineering

GitHub repository:

<https://github.com/RocketUp-Team/Fraud-Detection-Score-Risk>

Supervisor:

TAN Le Duy

© Copyright by RocketUpTeam 2026

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an academic fraud risk scoring workflow for the IEEE-CIS dataset. Apache Spark performs typed ingestion, transaction–identity joining, quality auditing, chronological leakage-controlled processing, feature engineering, and Parquet handover. Python training compares Logistic Regression, LightGBM, XGBoost, and CatBoost after the Spark-to-pandas boundary. The scoring package records preprocessing mappings, calibration state, thresholds, model metadata, and local explanations for an in-process FastAPI service with a React review interface. The current CatBoost balanced candidate achieves validation ROC-AUC 0.9081 and PR-AUC 0.5806, with holdout ROC-AUC 0.8781 and PR-AUC 0.4266. Because holdout PR-AUC is below the promotion target, the candidate remains not promoted and the configured serving default is unchanged. Offline–online feature mismatch, single-node training, fallback behaviour, and incomplete security and monitoring controls limit the result to an auditable academic demonstration rather than a production-readiness claim.

Acknowledgments

The members of RocketUpTeam sincerely thank supervisor TAN Le Duy for guidance and feedback provided throughout project development and documentation.

Team Roster and Repository Role Mapping

Table 1 lists the project team members, official roles, and primary repository responsibilities.

Table 1: *Team Roster and Repository Role Mapping*

Member Name	Role	Primary Responsibility
TUYEN Le Quang	Project Leader	Responsible for system ideation and overall project management.
NHI Nguyen Le Hong	Content & Presentation Coordinator	Responsible for synthesizing project content, designing slides, and developing the presentation strategy.
AN Duong Binh	Big Data & Pipeline Engineer	Responsible for Spark data ingestion, left joins, temporal splitting, and feature contract verification.
LONG Pham Duc	Frontend & UI Engineer	Responsible for React dashboard UI, risk visualizations, transaction queue, and analyst review workflow.
TRUNG Do Quoc	Backend & API Engineer	Responsible for FastAPI scoring service, database persistence, and scoring contracts.
QUAN Duong Hong	Machine Learning Engineer	Responsible for classifier training, hyperparameter tuning, SHAP explainability, isotonic calibration, and hold-out evaluation.

AI Assistance and Verification Disclosure

In accordance with academic integrity standards, the project team discloses the responsible use of Artificial Intelligence (AI) tools during initial project planning, code drafting, and documentation formatting.

The team explicitly affirms human ownership of the final work. AI-suggested code snippets, architectural ideas, and documentation drafts were inspected, edited, refactored, and verified by team members before inclusion. The Apache Spark ETL scripts, machine-learning estimators, isotonic calibration models, FastAPI endpoints, and React components were checked against the available runtime and test evidence rather than being accepted solely because an AI system proposed them. The metrics, PR-AUC and ROC-AUC values, hold-out results, checksums, and promotion-gate evidence reported in this document are tied to recorded execution artifacts and verified data contracts. This disclosure describes assistance and review; it does not replace the team's responsibility for the accuracy, originality, and academic integrity of the submitted report.

List of Abbreviations and Acronyms

Abbreviation	Full Definition / Meaning
API	Application Programming Interface
AUC	Area Under the Curve
BDA	Big Data Analytics
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
FPT	Corporation / FPT University
GBDT	Gradient Boosted Decision Trees
HTTP	Hypertext Transfer Protocol
IEEE-CIS	IEEE Computational Intelligence Society
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LR	Logistic Regression
ML	Machine Learning
MLlib	Machine Learning Library (Apache Spark)
PR-AUC	Precision-Recall Area Under the Curve
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
V2	Version 2 Model Release Authority

Contents

Abstract	3
Acknowledgments	4
List of Abbreviations and Acronyms	6
1 Introduction	14
1.1 Problem context	14
1.2 Objectives & research questions	14
1.3 BDA501 requirement compliance	15
1.4 Scope, contribution, and report organisation	17
2 Background and Related Work	19
2.1 Imbalanced temporal ranking	19
2.2 IEEE-CIS dataset structure	19
2.3 Dataset provenance and competition context	19
2.4 Raw files, records, and feature groups	20
2.5 Target definition and class distribution	21
2.6 Anonymisation, missingness, and reproducibility	21

2.7	Technology boundary	22
2.8	Why accuracy is insufficient for this dataset	22
2.9	Temporal leakage and operational realism	22
2.10	Interpretability as a review requirement	23
3	Requirements and System Architecture	24
3.1	Functional requirements	24
3.2	Quality requirements	24
3.3	Integrated architecture	25
3.4	System contracts	26
3.5	Technology stack used by the group	27
3.6	Implementation parameters and reproducibility	29
3.7	Architectural tradeoffs	30
3.8	Main source code components	31
3.9	Distributed Spark execution evidence	32
4	Big-Data Processing and Feature Engineering	34
4.1	Source validation and typed ingestion	34
4.2	Join integrity	35
4.3	Leakage-controlled transformation order	36
4.4	Chronological split	36

4.5	Feature engineering & class balance	37
4.6	Verification result	39
5	Model Development, Evaluation, and Serving	40
5.1	Artifact identity and terminology	40
5.2	Model-development protocol	40
5.3	PySpark MLlib DecisionTree baseline	42
5.4	Candidate models and selection	43
5.5	Current CatBoost candidate results	44
5.6	Packaging, checksum, and promotion	45
5.7	Request-time scoring	47
6	Application and Deployment	48
6.1	Integrated serving architecture	48
6.2	Scoring and review workflow	48
6.3	Deployment boundary	50
7	Validation and Whole-System Review	52
7.1	Evidence status	52
7.2	Reported validation results	52
7.3	Technical challenges and implemented solutions	55

7.4 Consolidated limitations	56
8 Conclusion and Future Work	58
8.1 Conclusion	58
8.2 Summary of research question answers	58
8.3 Future work	59
References	60
A API Reference	62
A.1 Common enums & core fields	62
A.2 Endpoint summary	62
B Deployment and Feature Contracts	64
B.1 Canonical feature groups	65
C Runbook and Role Mapping	66
C.1 Acceptance scenarios	66

List of Figures

3.1	Overall system architecture	26
3.2	Data, feature, and evaluation contracts	27
4.1	Detailed data processing pipeline	36
5.1	Model training lifecycle	42
5.2	Promotion state machine	47
6.1	Primary analyst views for triage and model-assisted risk interpretation. . . .	50
6.2	Detailed inspection and human-review actions supported by the application.	50
7.1	Dataset-level fraud composition and product-level fraud rate.	54
7.2	Exploratory transaction timing and candidate-model comparison.	54
7.3	Holdout evaluation evidence for ranking and classification behaviour. . . .	55
7.4	Probability calibration: raw versus isotonic-calibrated Brier score.	55
B.1	Default application deployment	64

List of Tables

1	Team Roster and Repository Role Mapping	4
1.1	BDA501 Requirement-Compliance Matrix	16
3.1	Functional requirements specification	24
3.2	Quality requirements specification	25
3.3	Implementation parameters and reproducibility settings	29
3.4	Major architectural tradeoffs	30
3.5	Main Source Files and Component Responsibilities	32
3.6	Distributed Spark Execution and Validation Evidence	33
4.1	Observed source files	34
4.2	Transaction–identity join audit	35
4.3	Chronological labeled splits	37
4.4	Training and evaluation class distributions	38
5.1	Model identifiers and runtime roles	40
5.2	PySpark MLlib DecisionTree Baseline Performance	43
5.3	Current model-family selection evidence	44
5.4	Current CatBoost candidate validation and holdout metrics	45

5.5	Current candidate promotion gate	46
7.1	Consolidated validation and reported project results	53
7.2	Technical Challenges, Implemented Solutions, and Evidence	56
A.1	Core API fields	62
A.2	Core REST Endpoints	63
B.1	Canonical feature groups	65
C.1	Acceptance scenarios	66

Chapter 1

Introduction

1.1 Problem context

Online transaction fraud is an imbalanced binary-classification problem in which the positive class is uncommon but operationally important. In the IEEE-CIS data used in this project, fraudulent transactions account for only a small fraction of all observations. This imbalance means that a model can appear accurate while failing to identify a meaningful share of fraud. It also creates asymmetric error costs: a false negative may allow a fraudulent transaction to proceed, whereas a false positive may send a legitimate transaction to manual review. The system is therefore evaluated as a ranking and decision-support system rather than through accuracy alone [1, 2].

How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring system?

1.2 Objectives & research questions

The first objective is to ingest and validate the **1.29 GB** IEEE-CIS dataset through Apache Spark so that the project can demonstrate an auditable big-data preparation process. The second objective is to create chronological, leakage-controlled `train`, `validation`, and `holdout` partitions. The third objective is to compare a Spark MLlib `DecisionTree` baseline with Logistic Regression, LightGBM, XGBoost, and CatBoost candidates using ranking-oriented metrics. The final objective is to package the selected scoring logic behind a FastAPI service, return TreeSHAP explanations, and expose the result through a React interface that supports analyst review.

The study is guided by four research questions. RQ1 asks whether the raw data can be processed while preserving transaction identity and preventing temporal leakage. RQ2 asks which classifier obtains the strongest validation PR-AUC and whether that ranking is retained on the later holdout period. RQ3 asks whether the packaged model can perform request-time scoring without requiring a live Spark dependency. RQ4 asks which architectural and operational limitations distinguish the academic demonstration from a production-ready fraud platform.

1.3 BDA501 requirement compliance

Table 1.1 summarizes compliance with BDA501 project rubric requirements.

Table 1.1: *BDA501 Requirement-Compliance Matrix*

Requirement	Project evidence
Main tool: Apache Spark	PySpark DataFrame & MLlib pipeline in Spark Preprocessing Module
Dataset size $\geq$ 500 MB	4 raw IEEE-CIS CSV files totaling 1.29 GB in raw dataset store
Raw-data ingestion	Explicit Spark schemas & required file validation in ingestion pipeline
Cleaning & transform	Stateless cleanup, identity normalization, missingness handling
Descriptive analysis	Spark <code>groupBy</code> , <code>agg</code> , Window analytics & EDA reporting module
Machine learning	PySpark MLlib DecisionTree baseline & CatBoost/LightGBM/XGBoost comparison
Distributed cluster	2 -worker Docker Compose Spark standalone cluster configuration
Architecture diagram	End-to-end system architecture & contract diagrams in documentation
Installation instructions	Project README and Docker Compose full-stack orchestration
Main source code	Modular source tree (pipeline, model, backend, frontend)
Analysis results	Saved JSON/Parquet metrics, PR-AUC, confusion matrix, calibration reports
Challenges & solutions	Summarized in Section 1.4 & Technical Challenges Matrix
Demo readiness	Full-stack Docker Compose deployment with React UI & FastAPI scoring backend

The compliance matrix shows that the project addresses the BDA501 requirements as an integrated workflow rather than as isolated demonstrations. The dataset size and Spark ingestion establish the big-data requirement, while the chronological split, model comparison, and saved metrics establish the analytical requirement. The architecture, source-code, and Docker rows show that the implementation is also packaged as a runnable system. The

matrix does not mean that every quality requirement is production-complete; it records the available project evidence and makes the remaining operational limitations explicit in later chapters.

1.4 Scope, contribution, and report organisation

The scope of this report is the complete evidence path from raw competition files to an analyst-facing risk decision. The report does not treat a model score as an isolated artefact. Instead, it evaluates whether the input data is preserved correctly, whether transformations are applied without future information, whether the model is compared on an appropriate imbalanced metric, and whether the resulting artefact can be loaded and explained by the serving application. This broader scope is important because a strong offline score is not useful if the online service receives a different feature representation or if an analyst cannot understand why a transaction was flagged.

The complete implementation, configuration files, and reproducibility artifacts are maintained in the project GitHub repository at <https://github.com/RocketUp-Team/Fraud-Detection-Score-Risk>. The principal contribution is an integrated, contract-oriented demonstration. The data pipeline records schema and feature-order assumptions, the training workflow separates model selection from holdout evaluation, and the serving layer exposes both a calibrated risk value and local feature attribution. The project also makes its negative evidence visible. In particular, the current CatBoost challenger is not promoted because its holdout PR-AUC misses the gate, and the application is explicitly bounded by missing online feature-store, security, and monitoring controls. Reporting these boundaries is part of the technical contribution because it prevents a demonstration result from being misinterpreted as a production claim.

The remainder of the report first establishes the fraud-detection setting and the dataset assumptions. It then describes the system contracts and integrated architecture, followed by the Spark data pipeline and feature-engineering decisions. The model chapter explains the comparison protocol and promotion decision. The application chapter describes request-

time scoring and analyst review, while the validation chapter brings together empirical evidence, challenges, and limitations before the conclusion identifies the next engineering steps.

Chapter 2

Background and Related Work

2.1 Imbalanced temporal ranking

Fraud detection is framed here as binary temporal ranking on rare positive cases, with an observed fraud rate of approximately **3.5%**. The model must rank risky transactions highly enough for investigators to review them efficiently, while the evaluation must reflect the fact that future transactions are not available when the model is trained. Chronological splitting along `TransactionDT` preserves that direction of time and reduces lookahead leakage. The resulting metrics should therefore be interpreted as evidence of performance on a later time window, not as proof that the model will behave identically under every future fraud pattern.

2.2 IEEE-CIS dataset structure

The dataset contains **590,540** labelled training transactions together with optional identity records [1]. Transaction attributes and identity attributes are stored separately, so joining them is a central data-engineering decision. The pipeline preserves the transaction table as the authoritative grain through a `LEFT JOIN`. This prevents transactions from disappearing merely because no identity record exists, while the resulting missingness is retained as information that can be used by the model and examined during quality analysis.

2.3 Dataset provenance and competition context

IEEE-CIS Fraud Detection was released as a Kaggle research prediction competition hosted by the IEEE Computational Intelligence Society in 2019. The competition asked participants to detect fraudulent customer transactions from structured transaction and identity information, and it attracted a large community of participants and teams [1]. The dataset

is useful for this project because it is large enough to require a deliberate data-processing strategy, contains both numerical and categorical variables, and represents a fraud setting in which the positive class is rare. It is also appropriate for studying the boundary between offline model development and online risk scoring because the transaction and identity information must be combined before a model can use the full available context.

The public dataset should not be interpreted as a complete representation of a bank's production data. The fields are anonymised, the original business definitions of many variables are unavailable, and the recorded competition period does not cover every geography, merchant population, or fraud pattern. Consequently, the project uses the data as a reproducible benchmark and engineering workload. Claims in this report concern the recorded benchmark and the implemented pipeline; they are not claims about the prevalence of fraud in the wider financial system.

2.4 Raw files, records, and feature groups

The raw release contains four principal files. The labelled transaction table contains **590,540** rows and 394 columns, including the target `isFraud`; the corresponding identity table contains **144,233** rows and 41 columns. The unlabelled test transaction table contains **506,691** rows and 393 columns, while the test identity table contains **141,907** rows and 41 columns. The project inventory records the four files as approximately **1.29 GB** in total. The difference between the number of transaction rows and identity rows is expected: identity information is available for only a subset of transactions, which is why a transaction- authoritative join is required.

The transaction table combines several conceptual groups. Core fields include `TransactionID`, `TransactionDT`, `TransactionAmt`, and `ProductCD`. Card and address fields describe payment-instrument and location-related patterns, while the C, D, M, and V families contain anonymised counts, time differences, matching signals, and engineered behavioural variables. The identity table adds fields related to device and digital identity, including variables such as `DeviceType`, `DeviceInfo`, and anonymised

identity attributes. Because the variable meanings are intentionally hidden, the pipeline uses their type, missingness, temporal position, and observed predictive behaviour without assigning unsupported business interpretations to individual columns.

2.5 Target definition and class distribution

The target variable is `isFraud`, where the positive value denotes a fraudulent transaction and the negative value denotes a legitimate transaction. The local dataset inventory reports **20,663** fraud transactions and **569,877** legitimate transactions, producing a fraud rate of **3.499%** and a legitimate-to-fraud ratio of approximately **27.58:1**. This ratio changes the meaning of every model metric. In particular, a model that predicts the majority class for almost all rows may appear accurate while being operationally useless. The project therefore preserves the natural class distribution in validation and holdout and uses weighting or controlled sampling only as an explicitly recorded training decision.

2.6 Anonymisation, missingness, and reproducibility

Anonymisation protects the original commercial data but limits the amount of domain interpretation that can be justified. A variable named `V123` cannot be described as a particular customer or device behaviour without an external data dictionary, so the report refers to feature families and measured effects rather than inventing semantic meanings. Missing values are also not automatically treated as noise. The absence of an identity record or the absence of a device attribute may be correlated with the way a transaction was collected. The project measures and preserves this missingness, while training-only imputation prevents information from the validation or holdout period leaking into the preprocessing state.

The dataset is therefore reproducible at the file, schema, split, and contract levels. The source inventory identifies the expected files and sizes, the pipeline records the join and split audits, and the model-ready export freezes the feature order and preprocessing metadata. Reproducing a numeric result still requires the same source release, software dependencies, configuration, and model artefacts; the public availability of the dataset alone does not guarantee identical results.

2.7 Technology boundary

The architecture deliberately separates **distributed Spark data preparation from single-node Python estimator training**. Spark is responsible for schema-aware ingestion, joins, temporal transformations, aggregate feature construction, and Parquet handover. After the model-ready contract is produced, Python-based estimators are trained and evaluated against the same frozen feature order. CatBoost, LightGBM, XGBoost, and Logistic Regression are compared, and TreeSHAP is used to provide local feature attributions for tree-model predictions [3]. This boundary satisfies the project requirement to use a big-data processing engine while making the current training-memory limitation explicit.

2.8 Why accuracy is insufficient for this dataset

Accuracy is a poor primary measure when legitimate transactions outnumber fraudulent transactions by roughly twenty-eight to one. A classifier that labels almost every row as legitimate can obtain a high accuracy value while detecting very little fraud. The report therefore gives priority to PR-AUC, which measures the precision-recall ranking behaviour under the rare-positive setting, and uses ROC-AUC as a complementary measure of class separation. Precision and recall at an operating threshold are reported because the review queue has a finite capacity and because investigators need to understand the trade-off between missed fraud and additional alerts. The Brier score is also retained for the calibrated candidate because a usable risk probability should be interpretable as more than an arbitrary ranking number.

2.9 Temporal leakage and operational realism

Temporal leakage occurs when information from a later period influences a training feature, preprocessing parameter, threshold, or model-selection decision. In a fraud system this can happen subtly: an entity aggregate may include transactions that happened after the row being scored, or an imputer may be fitted using the distribution of the holdout window. The project reduces this risk by ordering rows with `TransactionDT`, fitting learned state

on the earlier training window, selecting candidates on validation, and reporting final performance on a later holdout. This design is more demanding than a random split, but it produces evidence that is closer to the actual operating situation in which the model scores transactions arriving after its training history.

2.10 Interpretability as a review requirement

The system treats explainability as an operational requirement rather than a decorative visual. A fraud analyst needs to know which input characteristics contributed to a high score, especially when a legitimate customer may be temporarily blocked or escalated. TreeSHAP provides local attributions for tree-based models, allowing the interface to display the strongest positive and negative contributions for an individual transaction. These attributions explain the model's association with the observed feature vector; they do not prove that a feature caused the transaction to be fraudulent. That distinction is preserved in the report and in the application wording.

Chapter 3

Requirements and System Architecture

3.1 Functional requirements

Table 3.1: *Functional requirements specification*

ID	Requirement Description
FR-01	Validate and process required IEEE-CIS transaction and identity CSV files.
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.
FR-03	Preserve canonical feature order and preprocessing metadata.
FR-04	Compare binary classifiers using validation PR-AUC metrics.
FR-05	Score transactions and return probability, risk score, version, and explanations.
FR-06	Persist scored transactions and human review outcomes.
FR-07	Filter, sort, paginate, and inspect transaction records.
FR-08	Import CSV and load processed datasets with progress reporting.
FR-09	Support review approval/rejection and fraud/legitimate labels.
FR-10	Present risk, SHAP evidence, and model state in a browser interface.

The functional requirements define what a user can do with the system. They begin with validated data and model-ready outputs, then continue through scoring, persistence, filtering, import, and analyst review. This ordering reflects the dependency chain in the implementation: the review interface cannot provide a reliable decision unless the scoring contract and persisted transaction record exist first.

3.2 Quality requirements

Table 3.2: *Quality requirements specification*

ID	Requirement Description
QR-01	Temporal leakage control and non-overlapping chronological splits.
QR-02	Reproducible schema and feature contracts.
QR-03	Version-aware model artifacts and rollback capability.
QR-04	Explainable tree-model predictions via TreeSHAP.
QR-05	Local containerized deployment with bounded data mounts.
QR-06	Automated subsystem test suite.
QR-07	Auditable experiment and promotion lineage.
QR-08	Durable background job operations.
QR-09	Role-based authentication, authorization, and audit logging.
QR-10	Real-time production latency and drift monitoring.

The quality requirements define how the system should behave beyond the basic feature list. Temporal leakage control and contract verification protect the validity of the model result, while versioning, explainability, tests, and lineage protect reproducibility and reviewability. Durable jobs, access control, audit logging, latency monitoring, and drift monitoring are included as target requirements even where the current demonstration has not implemented them fully.

3.3 Integrated architecture

The system integrates batch data preparation, offline ML training, and online inference into an in-process modular monolith. Figure 3.1 presents the overall architecture and data flow contracts.

The architecture has four cooperating layers. The data-processing layer owns raw-file discovery, Spark transformations, feature engineering, and model-ready Parquet output. The model layer consumes that contract, trains and compares candidates, calibrates the selected challenger, and writes a versioned package. The backend layer loads the package, validates

scoring requests, persists results, and exposes typed endpoints. The frontend layer presents the risk decision, explanation, transaction history, and review action to a human user. The layers are logically decoupled through files and typed interfaces even though the current deployment packages several of them in one repository and local Compose environment.

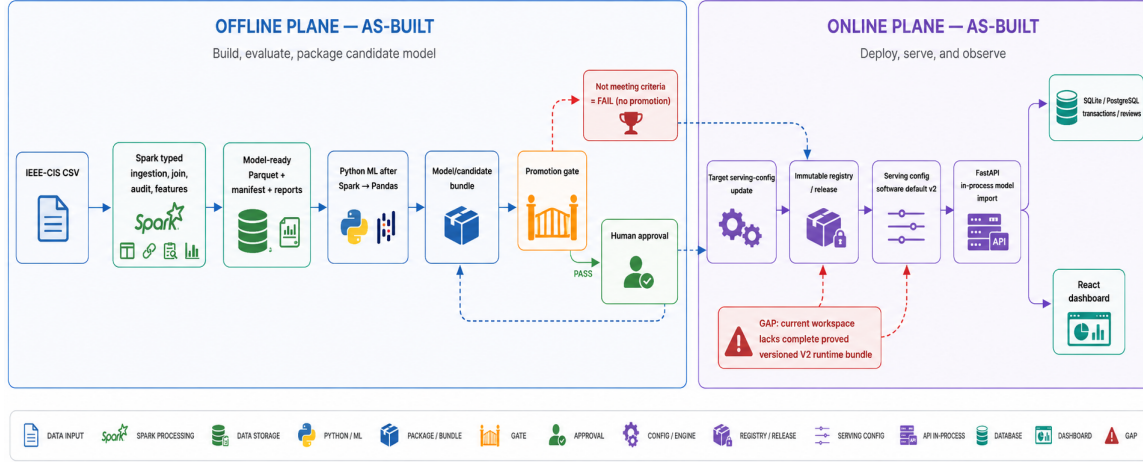


Figure 3.1: *Overall system architecture.*

3.4 System contracts

The system enforces three related contracts across its boundaries. The Data Contract defines the canonical 68-feature vector, its ordering, preprocessing metadata, and schema hash; this is the control that prevents the trainer and scorer from silently interpreting the same position as different variables. The Scoring Contract defines the per-transaction request and response, including the fraud probability, the derived risk score, and the TreeSHAP attribution payload. The API Contract then exposes those objects through typed Pydantic models on the backend and corresponding TypeScript types in the frontend. Together, these contracts make the handover between data preparation, model execution, and user interface explicit and testable.

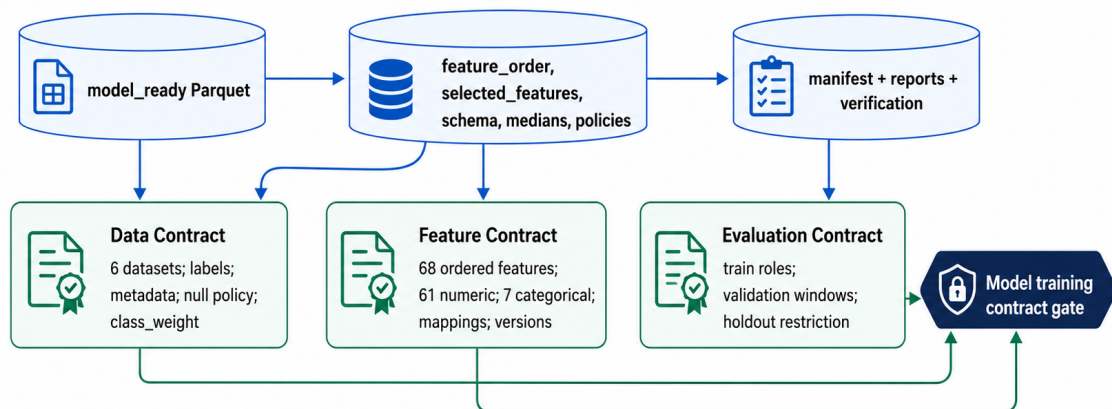


Figure 3.2: *Data, feature, and evaluation contracts.*

3.5 Technology stack used by the group

Apache Spark and PySpark form the data-processing foundation of the project. The group uses Spark DataFrames and Spark SQL-style transformations to read the wide CSV files with explicit schemas, validate identifiers, join transaction and identity records, calculate distributed summaries, construct temporal features, and export model-ready data. Spark MLlib is also used for the DecisionTree baseline, which gives the project a native distributed machine-learning reference in addition to the Python estimator experiments [4–6].

Parquet is used as the handover format between data preparation and model training. Its column-oriented representation preserves schema information and allows downstream jobs to read only the columns they need, which is more appropriate for repeated analytical access than passing the original CSV files between every stage [7]. The repository’s data contract adds the project-specific guarantees that Parquet alone cannot provide, including the canonical 68-feature order, imputation values, categorical mappings, and schema hash.

The modelling layer compares Logistic Regression, LightGBM, XGBoost, and CatBoost. Logistic Regression provides a linear reference, while the boosting families model nonlinear

interactions in a wide tabular feature space. CatBoost is particularly relevant because its documentation describes native handling of categorical features and feature combinations, although this project still passes a frozen contract to the estimator and records the exact preprocessing state used in its candidate package [8, 9]. The group uses isotonic calibration for the selected candidate so that the returned probability is evaluated separately from the ranking score. It uses SHAP/TreeSHAP at request time to expose local feature contributions; SHAP values are treated as additive explanations of the model output, not as causal proof [3].

FastAPI provides the backend HTTP service and Pydantic validation layer. The service loads the packaged model without starting Spark for every request, returns probability, risk score, policy band, decision, model version, and explanation fields, and persists scoring and review records through the database layer. PostgreSQL is the durable database target for the full stack, while SQLite is retained as a lightweight local configuration for development and demonstration.

React and TypeScript provide the browser dashboard. The group uses typed API responses to render a transaction queue, risk badges, score details, SHAP charts, import progress, and analyst review actions. This frontend is not a separate analytical model; it is the human-facing layer that turns the backend contract into a review workflow. Docker Compose packages the local services, including preprocessing, backend, database, and frontend profiles, so that the team can demonstrate the complete path in a controlled environment [10–13].

MLflow is used for experiment and model-lineage information in the training workflow. The recorded metadata helps distinguish a historical serving artifact from a current challenger, preserve calibration and threshold evidence, and evaluate promotion gates. The group deliberately keeps the promotion decision separate from the model-training library: an experiment can complete successfully while still failing the holdout-quality gate and remaining unpromoted [14].

3.6 Implementation parameters and reproducibility

Table 3.3 records the concrete implementation parameters used by the project. Version ranges are reported from the repository manifests, while values such as Spark memory, shuffle partitions, ratios, and random seed are taken from the pipeline configuration.

Table 3.3: *Implementation parameters and reproducibility settings*

Parameter	Recorded value and interpretation
Python runtime	Model and backend packages require Python 3.11 or newer; no single patch version is claimed.
Spark runtime	The model package and Docker training image use Apache Spark/PySpark 3.5.1 .
Data preparation	Spark <code>local[4]</code> , driver memory 8 GB , driver result limit 1 GB , shuffle partitions 64 , default parallelism 8 .
Cluster configuration	Standalone Spark master on port 7077 ; two workers, each configured with 2 cores and 4 GB memory.
Split policy	Chronological ratios of 70%/15%/15% for train, validation, and holdout.
Randomness	Imbalance sampling seed 42 ; model-family estimators use the same seed where supported.
Missingness policy	Missingness bands use thresholds 0.20 , 0.70 , and 0.95 ; quantile summaries use relative error 0.01 .
Serving stack	FastAPI/Uvicorn on port 8000 ; React/Vite on 5173 ; PostgreSQL 16 on 5432 ; optional Adminer on 8081 .
Feature handover	Frozen canonical vector of 68 features with preprocessing meta-data, categorical mappings, feature order, and schema hash.

The repository contains two dependency manifests with different PySpark constraints: the model package and preprocessing environment pin PySpark to **3.5.1**, whereas the generic root `requirements.txt` declares `pyspark>=4.2.0`. The controlled training and Docker path used for the reported results is the former **3.5.1** environment. This discrepancy

is retained as a reproducibility limitation and should be resolved before a clean installation is published as the single official environment.

The model manifest declares CatBoost **1.2.10** or newer, LightGBM **4.7.0** or newer, XGBoost **3.2.0** or newer, scikit-learn **1.9.0** or newer, SHAP **0.51.0** or newer, and MLflow **3.3.0** or newer. The backend manifest declares FastAPI **0.139.2** or newer, Uvicorn **0.51.0** or newer, SQLAlchemy **2.0.36** or newer, and psycpg **3.2.3** or newer. These are dependency constraints rather than a claim that every component was independently benchmarked at its latest release.

3.7 Architectural tradeoffs

Table 3.4: *Major architectural tradeoffs*

Choice	Benefit	Cost
Monorepo and path de-Simple handoff and one ComposeBackend inherits model dependencies	pendency stack	
Spark then pandas	Scalable preparation and familiar es-Driver-memory training ceiling	timators
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

The trade-off table shows that the current architecture favours clarity and local reproducibility over independent scaling. Spark provides a suitable processing boundary, but moving to a single-node Python estimator creates a memory ceiling. In-memory jobs and a heuristic fallback keep the demonstration available with minimal infrastructure, but they can lose work or conceal failures after a restart. These are deliberate compromises for the academic scope and are not presented as final production decisions.

These trade-offs reflect the project’s academic and demonstration objectives. Using Spark for preparation provides an appropriate distributed processing boundary and makes joins, windows, and aggregations explicit. Keeping the estimator stage in Python simplifies experimentation with several model families and calibration libraries, but it also means that the full training matrix must fit within the driver and model-training environment. Likewise, the modular monolith reduces integration overhead and makes a complete demo easy to run locally, but it does not provide the isolation, independent scaling, or failure containment expected from a production platform. The architecture is therefore intentionally useful for audit and demonstration, not presented as the final deployment topology.

3.8 Main source code components

Table 3.5 outlines the primary source implementation files and core responsibilities across project components.

Table 3.5: Main Source Files and Component Responsibilities

Component	Module reference	Responsibility
Spark Preprocessing	Preprocessing Pipeline	Schema ingestion, left join, leakage-free transformation, Parquet export
Data Verifier	Verification Engine	Validates dataset row counts, schema hash, split overlap, verification report
Model Training	Estimator Training Module	Single-node candidate training, tuning, calibration, holdout evaluation
Backend Scoring	FastAPI Scoring Service	Artifact loading, probability scoring, SHAP calculation, FastAPI endpoints
Frontend Dashboard	React UI Dashboard	React UI, risk queue, SHAP feature rendering, analyst review workflow
Docker Deployment	Multi-container Orchestration	Multi-container service orchestration (backend, frontend, db, preprocessing)

The source-component table maps repository responsibilities to the major subsystems. It shows that validation is implemented as its own evidence layer, rather than being hidden inside model training, and that the backend and frontend consume explicit scoring and API contracts. This separation makes a feature-pipeline change easier to check before it reaches the user interface.

3.9 Distributed Spark execution evidence

Table 3.6 presents the Spark execution configurations together with the validation evidence and reported outcomes. This expands the original infrastructure-only view so that the table records not only how the pipeline was configured, but also what was checked and what the run produced.

Table 3.6: *Distributed Spark Execution and Validation Evidence*

Stage	Configuration	Output	Validation and reported result
Local Spark preprocessing	local mode, one worker, four CPUs, 8GB memory	Model-ready Parquet and reports	94 checks passed and zero failed; source, schema, join, split, and feature-contract checks are ready for downstream training.
Configured cluster mode	Spark master on port 7077, two workers, four CPUs, 8GB memory	Cluster deployment specification	Configuration is implemented; a clean end-to-end distributed runtime was not independently verified.
Chronological validation	TransactionDT ordering from train to validation to holdout	Three labelled windows	Train: 412,956 ; validation: 88,486 ; holdout: 89,098 . Fraud rates are 3.5166% , 3.4310% , and 3.4849% .
Model-ready handover	Canonical 68 -feature vector and schema hash	Training contract	Handover is marked ready; no identifier or split-overlap failure was recorded.

Note: Cluster configuration is implemented, but a clean end-to-end distributed runtime was not independently verified.

Chapter 4

Big-Data Processing and Feature Engineering

4.1 Source validation and typed ingestion

The raw IEEE-CIS dataset contains four required CSV files totaling **1.29 GB**, as shown in Table 4.1. This size, combined with the wide schema and the need for repeated joins and aggregations, motivates the use of Spark rather than an ad hoc in-memory workflow. The ingestion stage applies explicit schemas instead of relying on automatic type inference, checks that every required file is present, and records the source inventory so that later results can be traced back to a known input state.

The source validation stage also treats the transaction identifier as a data quality control rather than merely a join key. Null identifiers and duplicate identifiers can change the grain of the dataset, create accidental row multiplication, or make a later prediction impossible to trace to its source. The pipeline therefore validates identifier presence and uniqueness before enrichment. It normalises the identity-side naming convention where needed so that train and test records follow the same join semantics. These checks are particularly important in an anonymised competition dataset because column names alone do not provide enough business meaning to detect a broken join.

Table 4.1: *Observed source files*

File	Bytes	Role
<code>train_transaction.csv</code>	683,351,067	Labeled transactions
<code>train_identity.csv</code>	26,529,680	Optional train identity
<code>test_transaction.csv</code>	613,194,934	Unlabeled transactions
<code>test_identity.csv</code>	25,797,161	Optional test identity

The source-file table demonstrates both the scale and the two-table structure of the workload. Transaction files dominate storage and row count, while the identity files are smaller but provide additional context for only a subset of transactions. This asymmetry explains why the pipeline treats transactions as the authoritative grain and preserves unmatched identity information.

4.2 Join integrity

Row authority is preserved through a LEFT JOIN on `TransactionID`, with the audit shown in Table 4.2. Every transaction row remains present after enrichment: the train join retains all **590,540** transactions and the test join retains all **506,691** transactions. The large unmatched counts are not treated as join failures because identity data is optional in the competition dataset. Instead, they are preserved as measurable missingness, allowing downstream analysis to distinguish an absent identity record from an invalid transaction key.

The join audit separates three cases that are often conflated in fraud pipelines. A transaction can have a valid identity match, can be a legitimate transaction with no identity record, or can reveal a genuine data-integrity problem such as a duplicated key. Only the third case should be treated as a join defect. Preserving the first two cases lets the model learn whether the presence or absence of identity evidence itself is informative, while the row-difference check ensures that the enrichment stage has not silently changed the number of transactions under evaluation.

Table 4.2: *Transaction–identity join audit*

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	590,540	144,233	446,307	0
Test	506,691	141,907	364,784	0

The join audit confirms that enrichment does not alter the transaction grain. There are **144,233** identity matches in train and **141,907** in test, but unmatched transactions remain

part of the dataset instead of being discarded. The zero row difference in both datasets is the key integrity result: missing identity information is represented as missingness, not as accidental loss of transactions.

4.3 Leakage-controlled transformation order

The transformation order is designed around a strict information boundary. Aggregate statistics, imputation values, categorical mappings, and any other learned preprocessing state are fitted only on the training partition. The frozen state is then applied forward to validation and holdout data without refitting. Figure 4.1 details this sequence. This is important because a random or leakage-prone preprocessing step could allow information from later transactions to influence the model before evaluation, producing metrics that are higher than the model’s likely future performance.



Figure 4.1: *Detailed data processing pipeline.*

4.4 Chronological split

Sorting by `TransactionDT` creates non-overlapping chronological splits, summarized in Table 4.3. The training window contains the earliest labelled transactions, the validation

window represents a later period used for model and threshold decisions, and the holdout window is reserved for the final evaluation. The fraud rates remain close to the overall rate across the three windows, but the time separation is still essential because fraud mechanisms, merchant behaviour, and missingness patterns can change over time.

Table 4.3: *Chronological labeled splits*

Dataset	Rows	Fraud	Fraud rate	TransactionDT range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,486	3,036	3.4310%	10,432,915–13,136,583
Holdout	89,098	3,105	3.4849%	13,136,627–15,811,131

The split table shows that the three evaluation windows are separated by `TransactionDT` and retain similar fraud rates. Training provides the historical learning window, validation is used for candidate and threshold decisions, and holdout represents later unseen data. Similar prevalence makes the comparison easier to interpret, but it does not remove the possibility of temporal feature or behaviour shift.

4.5 Feature engineering & class balance

The canonical 68-feature vector combines several types of evidence rather than depending on one raw column family. It includes transaction time and amount, indicators describing missingness, anonymized behavioural variables such as `C1--C14` and `D1--D15`, historical aggregates for repeated entities, and selected categorical attributes. Because fraud is rare, the training data is also represented in original, weighted, and controlled-balanced forms. The distribution in Table 4.4 shows why class weighting and sampling must be treated as modelling choices: they can improve the model’s attention to fraud, but they also change the precision-recall tradeoff and must be evaluated on the naturally imbalanced validation and holdout windows.

Historical entity features are constructed with a temporal constraint. For an entity such as a card, email, or device, the feature represents information available from earlier observations

rather than a summary that includes the current or future label. Examples include prior transaction counts, prior amount statistics, and the existence of previous activity. Such features can be highly useful in fraud detection because repeated behaviour and sudden velocity changes often distinguish risky activity. They are also a major source of leakage if calculated across the complete dataset before splitting, which is why the training-only and forward-application rules are central to the pipeline design.

Missingness is retained as a first-class signal. Numeric variables use values derived from the training partition for imputation, while categorical values can be represented through an explicit missing category. This avoids using validation or holdout distributions to determine preprocessing values and allows the model to distinguish an observed zero from an unavailable value. The resulting feature contract records the mappings and order needed by both training and request-time inference.

Table 4.4: *Training and evaluation class distributions*

Dataset	Legitimate	Fraud	Ratio
train_original	398,434	14,522	27.44
train_weighted	398,434	14,522	27.44
train_balanced	43,872	14,522	3.02
validation	85,450	3,036	28.15
holdout	85,993	3,105	27.70

The class-distribution table explains why the project evaluates weighting and sampling policies separately. Original and weighted datasets preserve the natural majority-to-fraud ratio, while the balanced dataset changes the training distribution. Validation and holdout remain naturally imbalanced, so the effect of a training policy must be judged by realistic ranking and review metrics rather than by training balance alone.

4.6 Verification result

The automated verifier passes **94** checks with zero failures. This result confirms that the exported data satisfies the expected schema, split, feature-order, and contract checks and can be handed to downstream model training. The status is recorded as:

```
ready_for_downstream_training
```

Chapter 5

Model Development, Evaluation, and Serving

5.1 Artifact identity and terminology

The project distinguishes historical evidence from the currently configured runtime artifact. Table 5.1 separates the Historical V2 LightGBM artifact, the LightGBM model that remains the configured serving default, and the current CatBoost Balanced challenger. This distinction is necessary because the strongest candidate in a new experiment must not be described as the serving champion until it passes the promotion gates.

Table 5.1: *Model identifiers and runtime roles*

Identifier	Model	Role	Status
Historical V2	LightGBM	Historical artifact	Historical
Serving default v2	LightGBM	Configured champion	Unchanged
Current candidate	CatBoost balanced	Challenger	Not promoted

The identifier table prevents three different concepts from being merged into one label. Historical V2 is an earlier LightGBM artifact, the serving default is the configured runtime champion, and the CatBoost Balanced model is only a current challenger. A candidate can win a validation comparison without becoming the model served by the application.

5.2 Model-development protocol

The training lifecycle follows the same boundary described in the architecture chapter: **Spark performs distributed data preparation, while Python estimators are fitted on a single node.** Figure 5.1 illustrates how model-ready artifacts are loaded, candidate families are trained, validation metrics guide selection, calibration and thresholds are recorded,

and the untouched holdout is used for final evidence. The workflow is reproducible at the level of contracts and saved metadata, although the single-node estimator stage remains a scalability boundary.

Model selection is intentionally performed before holdout evaluation. Candidate families are fitted using the training contract, compared on the validation window, and then subjected to calibration and operating-threshold decisions that are also based on information available before the holdout. The holdout is not used to tune hyperparameters or choose the winning family. This design prevents the final metric from becoming another training signal and makes the holdout result a more credible estimate of later-period behaviour. It also explains why the report distinguishes validation selection metrics from holdout operating metrics rather than presenting one undifferentiated score.

The comparison run uses **300** estimators or iterations for LightGBM, XGBoost, and CatBoost, with project seed **42**; Logistic Regression uses `max_iter=1000`. The validation tuning grid evaluates **200** and **400** estimators or iterations, tree depths **4** and **6**, and learning rates **0.05** and **0.10** for the boosting families. LightGBM is configured as deterministic and column-wise, XGBoost evaluates `aucpr`, and CatBoost uses a fixed random seed. These settings describe the implemented search space rather than claiming that every combination was equally successful.



Figure 5.1: *End-to-end model training lifecycle.*

5.3 PySpark MLlib DecisionTree baseline

To satisfy the BDA501 big-data machine-learning requirement, a PySpark MLlib `DecisionTreeClassifier` pipeline evaluates the 68-feature vector across original, weighted, and undersampled training variants. The results in Table 5.2 show the effect of imbalance treatment clearly. The unweighted tree produces very low PR-AUC because it is dominated by the majority class. Weighting increases recall and ranking quality substantially, but precision falls because the model raises more alerts. Undersampling changes the training distribution and improves neither ranking quality nor generalisation enough to replace the weighted baseline.

Table 5.2: *PySpark MLlib DecisionTree Baseline Performance*

Model variant	Split	PR-AUC	ROC-AUC	Precision	Recall	Time (s)
baseline_tree	Validation	0.0205	0.2329	0.6928	0.1574	13.5
baseline_tree	Holdout	0.0209	0.2366	0.6641	0.1948	13.5
weighted_tree	Validation	0.2666	0.6866	0.1564	0.6400	12.1
weighted_tree	Holdout	0.2773	0.6984	0.1494	0.6583	12.1
undersampled_tree	Validation	0.0284	0.4265	0.2791	0.5217	5.7
undersampled_tree	Holdout	0.0278	0.4163	0.2039	0.5076	5.7

The DecisionTree table provides a transparent baseline for the imbalance problem. The unweighted tree has high precision but very low recall and PR-AUC, which means that it identifies only a small fraction of fraud. Class weighting raises recall and ranking quality substantially, but precision decreases as the model produces more alerts. Undersampling changes the class ratio and reduces training time, yet its PR-AUC remains close to the weak baseline.

5.4 Candidate models and selection

Evaluation compares Logistic Regression, LightGBM, XGBoost, and CatBoost under the same downstream contract, as shown in Table 5.3. CatBoost Balanced achieves the highest validation PR-AUC among the listed candidates, narrowly exceeding weighted LightGBM while also retaining a strong ROC-AUC. This result supports selecting CatBoost Balanced as the current challenger, but it does not by itself justify replacing the serving default because model promotion depends on the later holdout and governance gates.

The comparison also illustrates why model choice cannot be based on a single metric or on model name. Logistic Regression provides a useful linear reference but underperforms the boosting candidates on the nonlinear and high-cardinality feature space. LightGBM and XGBoost provide strong ranking baselines, while CatBoost Balanced obtains the best validation PR-AUC in the recorded comparison. The selected candidate is therefore a result

of the observed contract-specific experiment, not a claim that CatBoost is always superior for fraud detection. A different temporal window, sampling policy, or feature contract could change the ordering.

Table 5.3: *Current model-family selection evidence*

Model	Training variant	ROC-AUC	PR-AUC	Decision
Logistic Regression	Weighted	0.8219	0.3090	Baseline
LightGBM	Weighted	0.9116	0.5682	Compared
XGBoost	Weighted	0.8879	0.5529	Compared
CatBoost	Weighted	0.8757	0.5245	Compared
CatBoost	Balanced	0.9070	0.5703	Selected

The model-family table shows that the linear reference is weaker than the boosting families on the recorded feature contract. LightGBM, XGBoost, and CatBoost capture nonlinear relationships more effectively, while CatBoost Balanced obtains the highest validation PR-AUC in this comparison. The table supports selecting it as a challenger, not claiming universal superiority.

5.5 Current CatBoost candidate results

Table 5.4 records the candidate across the validation and holdout windows. Validation results are used for model selection, while holdout results are treated as the more conservative estimate of performance on later data. The candidate reaches validation PR-AUC **0.5806** and ROC-AUC **0.9081**; on holdout, these decrease to **0.4266** and **0.8781**. The decrease is not evidence that the pipeline failed, but it is evidence that the candidate is sensitive to temporal shift and must be judged against the promotion threshold rather than the validation result alone.

At the selected holdout operating point, the candidate reports precision of **0.3820** and recall of **0.5182**. In practical terms, the review queue captures slightly more than half of the fraud cases represented by the holdout while keeping the flagged set substantially smaller

than the full transaction population. The trade-off is not universally optimal: a bank with high investigation capacity may prefer higher recall and tolerate more false alerts, while a constrained team may require higher precision. The report therefore treats the threshold as a policy decision that must remain visible and configurable, rather than as a permanent property of the model weights.

Table 5.4: *Current CatBoost candidate validation and holdout metrics*

Window	Threshold	PR-AUC	ROC-AUC	Precision	Recall	Brier
Validation selection	0.50	0.5806	0.9081	0.3970	0.6220	–
Holdout	0.16	0.4266	0.8781	0.3820	0.5182	0.0248

The candidate-metrics table separates selection evidence from final evidence. Validation results justify further evaluation, but the holdout decline in PR-AUC and ROC-AUC shows that later-period behaviour is more difficult. Precision and recall describe the practical review trade-off, while the Brier score records probability reliability after calibration. The columns should therefore be read together rather than reduced to one headline metric.

5.6 Packaging, checksum, and promotion

The candidate promotion gate evaluates nine criteria, summarized in Table 5.5. The decisive failure is holdout PR-AUC: the observed **0.4266** is below the target **0.4582**. Although holdout ROC-AUC, review precision, calibration non-regression, checksum integrity, artifact loading, and data-contract checks pass, those successes do not override the failed ranking-quality gate. The candidate therefore remains `not_promoted`, and the configured serving default is unchanged. This conservative decision keeps experimental performance claims separate from runtime behaviour.

The promotion state also demonstrates the value of separating technical validity from model quality. A checksum match proves that the expected file was loaded, but it does not prove that the model is good. A passing schema gate proves that the feature vector is compatible, but it does not prove that the candidate generalises. Conversely, a failed performance gate

does not mean that the data contract or serving package is broken. The promotion process keeps these questions separate so that the team can improve the candidate or revise the target without losing confidence in the integrity of the serving default.

Table 5.5: *Current candidate promotion gate*

Gate	Result	Evidence
Holdout PR-AUC ≥ 0.4582	Fail	0.4266
Holdout ROC-AUC ≥ 0.8746	Pass	0.8781
Review precision ≥ 0.30	Pass	0.3820
Calibration non-regression	Pass	Brier decreased
Artifact checksum	Pass	SHA-256 match
Artifact-load smoke	Pass	Manifest verified
Data contract verification	Pass	94 checks ready
Version match	Pass	v2
Git clean	Fail	git_dirty=true

The promotion-gate table demonstrates that technical integrity and model quality are different checks. Checksums, artifact loading, the data contract, version matching, ROC-AUC, precision, and calibration pass, but holdout PR-AUC fails because **0.4266** is below **0.4582**. The candidate remains unpromoted and the serving default is unchanged; this is a controlled release decision rather than a failure of the entire pipeline.

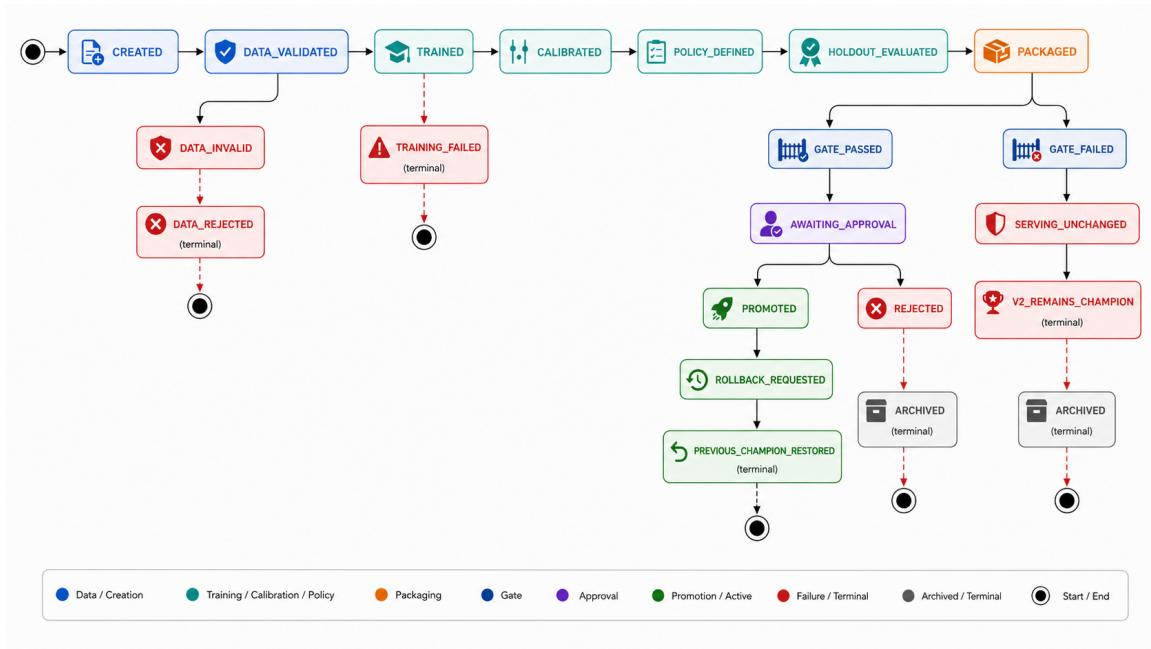


Figure 5.2: *Promotion state machine.*

5.7 Request-time scoring

At request time, the scoring package maps categorical values using persisted indexers, applies the saved preprocessing and isotonic calibration state, and computes TreeSHAP feature attributions [3]. The API supports both `full_feature` and `partial_demo` modes so that the demonstration can score complete model-ready records while still showing the user-interface flow for reduced examples. This packaging removes the need to start Spark for each request, but it also means that online feature construction must remain aligned with the offline contract if the system is ever deployed beyond the demonstration setting.

Chapter 6

Application and Deployment

6.1 Integrated serving architecture

The application operates as an **in-process model-serving modular monolith**. FastAPI imports the scoring package directly, which keeps the request path simple and avoids a separate model-serving network hop. Scored transactions are persisted through PostgreSQL or the local SQLite configuration, while the React dashboard consumes typed API responses. This arrangement is appropriate for a controlled academic demonstration because the data contract, scoring logic, persistence layer, and user interface can be exercised together; however, it also concentrates several responsibilities in one deployable unit and therefore leaves scaling, isolation, and operational fault handling for future work.

The request path begins when a transaction is submitted or imported. The API validates the request shape, maps the available fields to the canonical feature order, invokes the packaged model, applies calibration and policy mapping, and persists the resulting score. The response then carries both machine-oriented values, such as probability and model version, and human-oriented values, such as risk band, decision, and feature explanation. Keeping these values together makes it possible to reproduce what the analyst saw at the time of review and to distinguish an old score from a score produced after a model update.

6.2 Scoring and review workflow

For each scored transaction, the backend returns a calibrated probability, a risk score on a **0–100** scale, a risk band, an operational decision, and TreeSHAP attributions. The probability expresses the model output, whereas the risk band and decision are policy-facing interpretations and should not be confused with the learned model itself. Scored transactions and human analyst reviews are stored as distinct database entities, allowing the system to pre-

serve both the automated assessment and the later human outcome. The interface figures in this section show how the monitoring queue, transaction detail, explanation panel, and review action form one analyst workflow.

The operating policy is explicit in the packaged V2 artifact. The review threshold is fraud probability **0.16**, and the reject threshold is **0.53**. The review threshold is selected by maximising recall and then F1 subject to minimum precision **0.30**; the reject threshold is the lowest threshold meeting minimum reject precision **0.60**. After conversion to a risk score, scores **0–19** are low and approve, **20–39** are guarded and approve, **40–59** are medium and review, **60–79** are high and review, and **80–100** are critical and reject. This policy is separate from model ranking and can be changed through a governed policy update without changing the learned weights.

The review workflow is designed as a human-in-the-loop process. A high-risk score creates a prioritisation signal, not an automatic declaration of fraud. An analyst can inspect the transaction context and the most influential features, then record a review outcome. Separating the automated score from the human review is valuable for later evaluation because analyst labels can be used to examine false positives, missed fraud, and threshold performance. It also avoids overwriting the original model output when a reviewer disagrees with the prediction.

(a) Transaction monitoring queue.

(b) Risk score and SHAP feature importance.

Figure 6.1: *Primary analyst views for triage and model-assisted risk interpretation.*

(a) *Transaction detail panel.*

(b) Analyst fraud review workflow.

Figure 6.2: *Detailed inspection and human-review actions supported by the application.*

6.3 Deployment boundary

Docker Compose orchestrates the preprocessing, training, backend, and React UI profiles, with processed Parquet artifacts mounted into the backend container. This provides a repeatable local deployment path and makes the subsystem boundaries visible to evaluators. It should not be interpreted as evidence of production availability: the current deployment still relies on local configuration, limited background-job durability, incomplete security

controls, and a fallback path whose operational behaviour requires stronger monitoring.

The deployment boundary is therefore deliberately local and demonstrable. It supports repeatable evaluation of the data contract, model package, API responses, persistence path, and user-interface flow on one controlled environment. It does not yet establish multi-instance coordination, durable queue processing, secret management, service-level latency, or safe recovery from partial failure. Those properties require an operational environment and test evidence beyond the current academic scope.

No formal latency benchmark or throughput target is claimed in this report. The repository includes API tests and a warm-up path for model loading, but request-time p50/p95 latency, concurrent-request capacity, and end-to-end batch throughput require a controlled performance test on the target hardware. This distinction prevents the port configuration and successful functional response from being misread as production performance evidence.

Chapter 7

Validation and Whole-System Review

7.1 Evidence status

The verifier passes **94** validation checks and records the status `ready_for_downstream_training`. The saved artifacts provide empirical evidence for the data contract, split integrity, baseline comparisons, and candidate holdout evaluation. This is a statement about the consistency and readiness of the recorded workflow; it is not a statement that the current candidate has passed the promotion gate or that the overall application is production-ready.

The validation evidence should be read at three levels. At the data level, the pipeline verifies row preservation, schema consistency, split separation, and feature-order stability. At the modelling level, the saved comparisons show how imbalance treatment and model family affect ranking and operating metrics. At the application level, the packaged artefact can be loaded and mapped to the scoring contract. The evidence is strongest for these recorded local runs; it is weaker for long-running production concerns such as drift, availability, access control, and performance under concurrent traffic.

7.2 Reported validation results

The accompanying project presentation and the repository artifacts report a consistent validation story. At the data-processing level, the verifier passes **94** checks with zero failures and produces the status `ready_for_downstream_training`. The transaction–identity audit preserves all **590,540** training transactions and all **506,691** test transactions, while the chronological split produces **412,956** training, **88,486** validation, and **89,098** holdout rows. The fraud rates in the three windows remain close to the overall rate, which supports comparison without replacing the time-ordered evaluation design.

At the model-selection level, CatBoost Balanced is the current candidate after achieving the strongest recorded validation result in the model-family comparison. Its validation PR-AUC is **0.5806** and validation ROC-AUC is **0.9081**. These values support candidate selection, but they are not used as a release claim by themselves. The holdout evaluation is deliberately reported separately: holdout PR-AUC is **0.4266**, holdout ROC-AUC is **0.8781**, precision is **0.3820**, recall is **0.5182**, and the post-calibration Brier score is **0.0248**. The difference between validation and holdout PR-AUC is the central reason the candidate remains unpromoted.

At the application level, the project reports that the backend can return a calibrated fraud probability, a 0–100 risk score, a risk band, a decision, the model version, and TreeSHAP attributions. The API supports both `full_feature` and `partial_demo` scoring modes, while the React review layer presents the score and explanation for analyst inspection. This evidence demonstrates an integrated serving path; it does not establish production latency, online feature parity, or autonomous decision safety.

Table 7.1: *Consolidated validation and reported project results*

Validation layer	Reported result	Interpretation
Data contract	94 passed, 0 failed	Model-ready handover is structurally valid and traceable.
Chronological split	412,956/88,486/89,098 rows	Time-ordered train, validation, and holdout windows are non-overlapping.
Candidate selection	Validation PR-AUC 0.5806 ; ROC-AUC 0.9081	CatBoost Balanced is the current challenger.
Holdout evaluation	PR-AUC 0.4266 ; ROC-AUC 0.8781	Later-period ranking quality falls below the promotion target.
Operating point	Precision 0.3820 ; recall 0.5182 ; Brier 0.0248	The candidate provides a usable review trade-off, but requires policy and monitoring controls.
Promotion decision	<code>not_promoted</code>	Holdout PR-AUC is below target 0.4582 ; the serving default is unchanged.

The consolidated-results table provides one reading path across the system. The first two rows establish that the data contract and chronological windows are valid. The next rows show that CatBoost Balanced is strong during validation but weaker on holdout, and that its operating threshold produces a precision-recall trade-off suitable for review rather than blind automation. The final row records the governance consequence: the candidate remains unpromoted because the primary holdout ranking gate is not met.

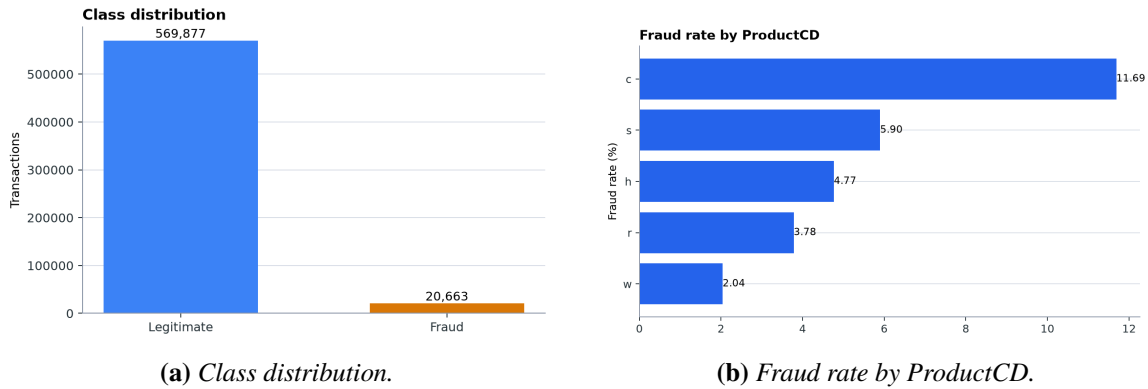


Figure 7.1: Dataset-level fraud composition and product-level fraud rate.

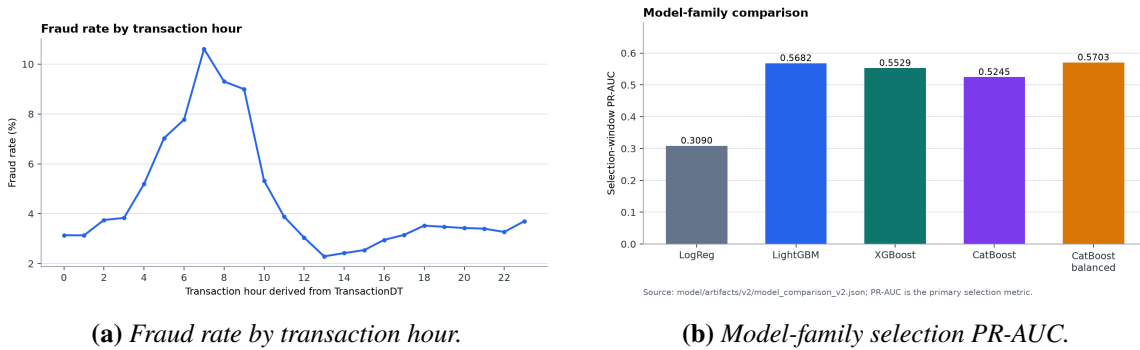
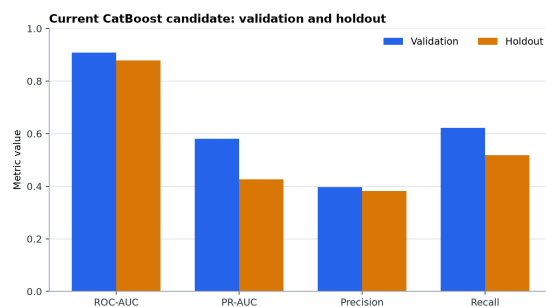
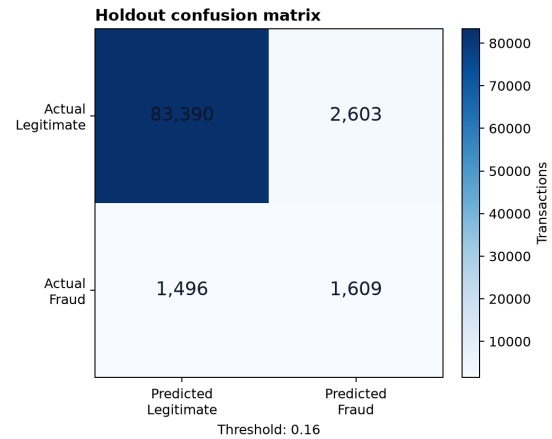


Figure 7.2: Exploratory transaction timing and candidate-model comparison.



(a) Validation versus holdout metrics.



(b) Holdout confusion matrix.

Figure 7.3: Holdout evaluation evidence for ranking and classification behaviour.

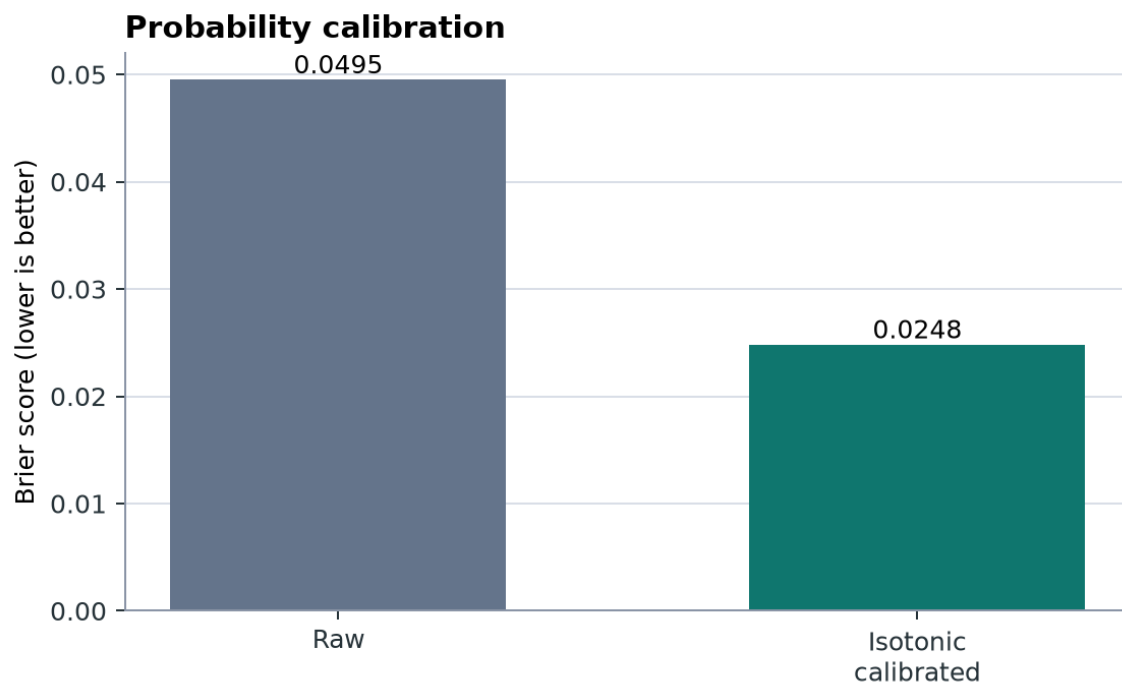


Figure 7.4: Probability calibration: raw versus isotonic-calibrated Brier score.

7.3 Technical challenges and implemented solutions

Table 7.2 details the technical challenges, solutions, and repository evidence.

Table 7.2: *Technical Challenges, Implemented Solutions, and Evidence*

Challenge	Implemented solution	Evidence
Transaction–identity sparsity	Transaction-authoritative LEFT JOIN pre-serving 100% transaction rows with missingness indicators	Preprocessing Join Audit Step
Missing values	Train-only median imputation for numeric; explicit 'missing' category	Feature Contract Specification
Class imbalance	Evaluated original, weighted (weight 3.0), and balanced undersampled splits	Balanced Parquet Artifact
Temporal leakage	Chronological split on TransactionDT (train → val → calib → holdout)	Leakage-free Pipeline Stages
Schema & feature order	Frozen canonical 68-feature vector and machine-checked schema hash gate	Verification Engine Gate
Verifier failure	Automated validation gate outputting ready_for_downstream_training	Verification Report Artifact
Spark-to-pandas boundary	Spark handles distributed data preparation; Python fits single-node matrices	Model Execution Logs
Offline–online feature mismatch	Standardized backend feature loader mapping scoring request vector to schema	Backend Feature Loader

The challenges table connects each technical risk to an implemented control and an evidence location. The controls are layered: row preservation protects the join, train-only transformations protect against leakage, the frozen feature vector protects compatibility, and the backend loader protects request-time mapping. The table also shows where the controls stop, since offline–online mismatch and the Spark-to-pandas boundary remain known limitations.

7.4 Consolidated limitations

The first limitation concerns reproducibility at scale: experiment tracking is local and the estimator stage uses single-node pandas-compatible matrices, so the workflow has a clear

memory and operational ceiling. The second concerns policy alignment: backend risk bands are configured separately from offline threshold optimisation, which can cause the displayed decision policy and the evaluated operating point to diverge. The third concerns offline–online parity because a real-time feature store for historical aggregates has not been implemented; the API therefore cannot yet guarantee that every online aggregate is calculated in exactly the same way as its training counterpart. The fourth concerns operational controls, since the demonstration includes a fail-open heuristic and in-memory background jobs that could conceal model-loading failures or lose work after restart. The fifth concerns security and governance: authentication, role-based authorization, and durable audit logging are not complete, so the system should not be used to make unsupervised real-world decisions.

These limitations affect how the results should be used. The model metrics are appropriate for comparing the recorded candidates and for demonstrating a controlled validation procedure. They are not sufficient to estimate fraud loss reduction, customer impact, or regulatory suitability in a live business context. Before operational use, the project would need an online feature parity test, a policy calibration study with real review capacity, security testing, durable audit trails, drift monitoring, rollback procedures, and repeated evaluation on later data. The explicit limitation statement is thus a guardrail around interpretation, not a rejection of the current results.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

The project demonstrates an end-to-end fraud scoring workflow that separates distributed Spark data preparation from single-node Python model training and in-process API serving. The strongest current challenger, CatBoost Balanced, achieves validation ROC-AUC **0.9081** and PR-AUC **0.5806**; on the later holdout it achieves ROC-AUC **0.8781** and PR-AUC **0.4266**. Because holdout PR-AUC is below the required promotion target, the candidate remains `not_promoted`. The most defensible conclusion is therefore that the project has produced a coherent and auditable academic demonstration, not a production-ready fraud decision system.

The main lesson is that fraud-risk quality depends on the relationship between data, time, model, policy, and review workflow. The Spark pipeline protects the data boundary and produces a reusable contract, while the model workflow provides a disciplined comparison between candidate families. Calibration, TreeSHAP explanations, and analyst review make the score more usable, but they do not remove the need for monitoring and governance. The failed holdout promotion gate is particularly informative: it shows that a candidate can be technically valid, explainable, and strong on validation while still requiring more evidence before becoming the runtime default.

8.2 Summary of research question answers

The answer to RQ1 is positive within the recorded evidence: chronological splits, schema contracts, and the automated verification checks support a leakage-controlled handover. The answer to RQ2 is qualified: CatBoost Balanced obtains the highest validation PR-AUC among the current candidates, but its holdout PR-AUC falls below the promotion target, so

validation leadership is not sufficient for promotion. The answer to RQ3 is also positive for the demonstration: the in-process Python package can score requests without a Spark dependency at request time. Finally, RQ4 is answered by the limitations documented in the validation chapter: offline–online feature parity, policy ownership, fallback behaviour, security, and monitoring define the boundary between the current demo and a deployable service.

8.3 Future work

The first priority is to implement a real-time feature store, for example with Feast or a Redis-backed stream-processing layer, so that rolling entity aggregates can be calculated consistently at inference time. The second priority is to separate operational policy from model weights by introducing a dynamic threshold and risk-band service that authorised risk officers can update without redeploying the model artifact. The third priority is to add MLOps controls for population stability, prediction drift, experiment lineage, and controlled retraining when validated performance falls below an agreed service level. A later research direction is to model relationships between transactions, devices, cards, and identities with graph neural networks and temporal sequence models, since coordinated fraud may not be visible in isolated rows. Finally, the analyst experience can be strengthened by extending TreeSHAP with actionable counterfactual explanations that clarify which measurable conditions would change a review outcome.

The recommended implementation order follows dependency risk. Online feature parity should be addressed first because a stronger model cannot compensate for features that are calculated differently at serving time. Policy ownership and audit logging should follow so that decisions are traceable and thresholds can be changed safely. Drift monitoring and automated retraining should then be introduced only after the measurement and rollback procedures are stable. Graph and sequence models are research extensions rather than immediate replacement candidates; they should be evaluated against the same chronological holdout and promotion gates. This sequence keeps future complexity tied to measurable operational benefit.

Bibliography

- [1] IEEE Computational Intelligence Society, “Ieee-cis fraud detection,” 2019.
- [2] Amazon Science, “Fraud dataset benchmark: Ieee-cis fraud detection,” 2023.
- [3] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [5] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, R. Xin, R. Xin, M. J. Franklin, R. R. Zadeh, M. Zaharia, and A. Talwalkar, “Mllib: Machine learning in apache spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [6] Apache Software Foundation, “Mllib: Main guide,” 2026.
- [7] Apache Software Foundation, “Parquet files,” 2026.
- [8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, vol. 31, pp. 6638–6648, 2018.
- [9] CatBoost Developers, “Categorical features and shapvalues,” 2026.
- [10] S. Ramírez, “Fastapi documentation.”
- [11] PostgreSQL Global Development Group, “Postgresql documentation.”
- [12] Meta Open Source, “React documentation.”

[13] Docker Inc., “Docker documentation,” 2026.

[14] MLflow Contributors, “Mlflow documentation.”

Chapter A

API Reference

A.1 Common enums & core fields

```
RiskBand = low | guarded | medium | high | critical  
Decision = approve | review | reject
```

Table A.1: *Core API fields*

Field	Type	Meaning
fraud_probability	float	Model output in $[0, 1]$
risk_score	integer	Probability $\times 100$
risk_band	enum	Backend score band
decision	enum	Policy result (approve/review/reject)
scoring_mode	enum	full_feature or partial_demo
shap_top5	list/null	Local TreeSHAP contributions

The core-field table defines the shared vocabulary between the scoring service and the dashboard. Probability is kept distinct from the display score, while risk band and decision represent policy interpretation rather than raw model output. The scoring mode records whether the request used the complete contract or the reduced demonstration path, and the SHAP field carries local explanation evidence for analyst review.

A.2 Endpoint summary

The local reference deployment exposes the API through FastAPI/Uvicorn at `http://localhost:8000`; the browser dashboard uses port **5173**, and PostgreSQL uses port

5432. The endpoint table records the request surface used for health checks, metadata inspection, scoring, import, dataset loading, transaction review, and asynchronous job monitoring.

Table A.2: *Core REST Endpoints*

Method	Path	Response
GET	/health	Process health status
GET	/meta	Model & schema metadata
POST	/score	Risk score response
GET	/transactions	Paginated transaction queue
GET	/transactions/{id}	Detailed transaction record
POST	/transactions/{id}/review	Analyst review decision update

The endpoint table groups the API by the user’s workflow. Health and metadata endpoints establish service and model state, scoring creates a risk assessment, transaction endpoints support queue inspection, and the review endpoint records the human outcome. This mapping makes it possible to trace a dashboard action back to a specific backend responsibility.

Chapter B

Deployment and Feature Contracts

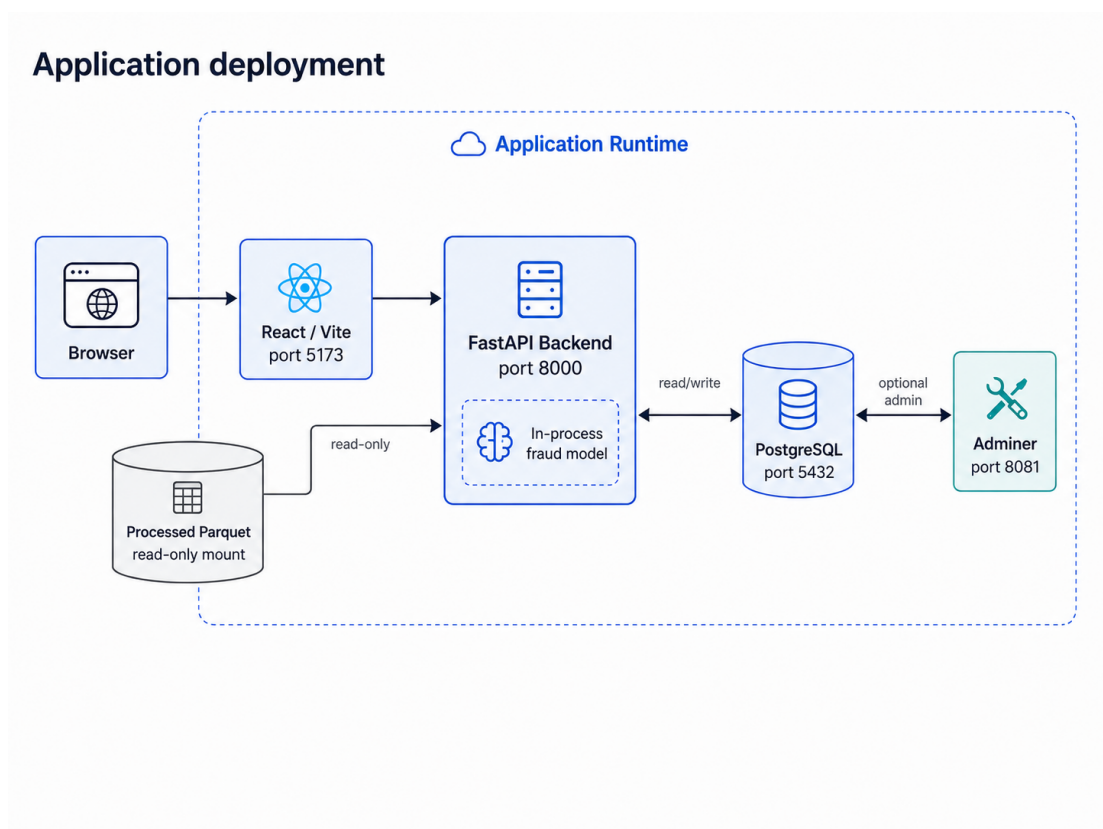


Figure B.1: Default application deployment. Browser, React/Vite, FastAPI backend, PostgreSQL, and in-process fraud model.

B.1 Canonical feature groups

Table B.1: *Canonical feature groups*

Group	Representative features
Amount	TransactionAmt, log/capped/decimal amount, card-mean ratios
Time	TransactionDT, day, week, hour, day-of-week, age, night flag
Missingness	Selected identity/device missing counts and ratios
Presence	Identity, device, email domain, distance, address flags
Behavior	dist1, dist2, selected C1--C14, selected D1--D15
Categorical	ProductCD, card4, card6, DeviceType, device_family, M4

The feature-group table explains how the canonical vector is organised. Amount and time describe the immediate transaction, missingness and presence features describe the availability of identity evidence, behaviour features capture anonymised patterns, and categorical fields preserve selected discrete signals. The groups are explanatory; the model still consumes the exact frozen feature order defined by the data contract.

Chapter C

Runbook and Role Mapping

C.1 Acceptance scenarios

Table C.1: *Acceptance scenarios*

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	<code>/meta</code> returns expected version with no warning
Full-feature score	<code>scoring_mode=full_feature</code> ; probability bounded
Partial score	<code>partial_demo</code> is visible; required fields enforced
Review update	Status/label persist and queue/statistics invalidate

The acceptance table converts the system requirements into repeatable smoke tests. A successful handoff, model startup, full or partial score, and review update together cover the main path from stored artefact to analyst action. These scenarios demonstrate functional integration, but they do not replace load testing, security testing, drift monitoring, or production recovery tests.